FCA · UNAM — SISTEMA SUIF

Instalación del entorno de desarrollo

VirtualBox + AlmaLinux 10 mínimo · Apache · PHP 8.4 · PostgreSQL 18 · Laravel 13

Guía paso a paso para que cada integrante del equipo monte en su computadora con Windows una máquina virtual con SUIF funcionando, conectada a GitHub con su propia cuenta y lista para desarrollar y correr las pruebas.

Documento	Guía de instalación del entorno de desarrollo (máquina virtual)
Dirigido a	Integrantes del equipo de desarrollo de SUIF
Repositorio	github.com/SUIF-Team/SUIF
Equipo anfitrión	Windows 10/11 de 64 bits con VirtualBox 7.2
Sistema invitado	AlmaLinux 10.2 (ISO mínima, x86_64)
Fecha	14 de octubre de 2026 · versión 1.1 (base de datos con un solo archivo de instalación)
Sustituye	A la guía anterior *SUIF-Instalacion-Entorno-AlmaLinux10.docx* en lo que toca a la VM de desarrollo. El server de despliegue sigue su propio procedimiento en *deploy/README.md*.

Contenido

1 Antes de empezar: lo que debes tener
2 Cómo queda el entorno
3 Tus datos personales
4 Paso 1 — Descargas
5 Paso 2 — Instalar VirtualBox
6 Paso 3 — Crear la máquina virtual
7 Paso 4 — Instalar AlmaLinux 10
8 Paso 5 — Entrar por SSH desde Windows
9 Paso 6 — Cargar tus variables
10 Paso 7 — Sistema base y cortafuegos
11 Paso 8 — Repositorios de paquetes
12 Paso 9 — PHP 8.4 y Apache
13 Paso 10 — Composer y Node.js
14 Paso 11 — PostgreSQL 18
15 Paso 12 — Rol, base de datos y .pgpass
16 Paso 13 — Git y tu llave SSH de GitHub
17 Paso 14 — Clonar SUIF, permisos y SELinux
18 Paso 15 — Sitio de Apache
19 Paso 16 — Archivo .env y dependencias
20 Paso 17 — Esquema de la base de datos
21 Paso 18 — Administrador y datos de prueba
22 Paso 19 — Verificación final
23 Trabajo diario
A Anexo A — Problemas conocidos
B Anexo B — Diagnóstico rápido

CÓMO LEER ESTA GUÍA

Cada bloque de comandos indica arriba **dónde** se ejecuta: **Windows · PowerShell** o **VM · terminal SSH**. Los textos en `amarillo` dentro del código son valores que cambias por los tuyos. Cada paso termina con una comprobación: **no avances si el resultado no coincide**; busca el síntoma en el Anexo A.

POR QUÉ ESTA GUÍA ES DISTINTA DE LA ANTERIOR

La guía anterior era para el server y arrastraba tres fallas que explican casi todos los problemas con PostgreSQL:

• Pedía `dnf module disable postgresql`, que en AlmaLinux 10 ya no existe y termina en error (PGDG sólo lo pide en EL8).
• Cargaba 3 de los 12 scripts de la base de datos, y los archivos «completos» del repositorio estaban desfasados.
• Corría `psql` con contraseña tecleada a mano en cada comando.

Aquí la base se carga con un solo archivo, `database/scripts/instalacion.sql`, que ya trae el esquema completo. La contraseña se genera una vez y se guarda en `~/.pgpass`.

1. Antes de empezar: lo que debes tener

Revisa esta lista completa antes de descargar nada. Lo que falte aquí es lo que después aparece como un error difícil de entender.

Cuentas y accesos

- [ ] **Cuenta de GitHub** propia, con correo verificado. Si no la tienes: github.com/signup.
- [ ] **Invitación al repositorio aceptada.** Pide al responsable del equipo que te agregue como colaborador de SUIF-Team/SUIF y acepta el correo de invitación (o entra a github.com/SUIF-Team/SUIF/invitations). Sin esto puedes clonar, pero `git push` te responde *Permission denied*.
- [ ] **Verificación en dos pasos** activada en GitHub (recomendado; la organización puede exigirla).
- [ ] **Permisos de administrador** en tu Windows, para instalar VirtualBox.

Equipo (anfitrión)

- [ ] **Windows 10 u 11 de 64 bits**.
- [ ] **8 GB de RAM** o más (la VM usa 4 GB).
- [ ] **40 GB libres** en disco, de preferencia SSD.
- [ ] **Virtualización activada** en el BIOS/UEFI (Intel VT-x o AMD-V). Compruébalo en *Administrador de tareas → Rendimiento → CPU → Virtualización: Habilitado*.
- [ ] **Internet** sin proxy que bloquee GitHub, nodejs.org, getcomposer.org o los repositorios de AlmaLinux, Remi y PostgreSQL.

Aplicaciones en Windows

Aplicación	¿Obligatoria?	Para qué
Oracle VirtualBox 7.2	Sí	Ejecuta la máquina virtual. Paso 2.
Microsoft Visual C++ Redistributable (x64)	Sólo si VirtualBox lo pide	Requisito del instalador de VirtualBox en algunos equipos.
Terminal de Windows / PowerShell	Sí (ya viene)	Trae el cliente `ssh` con el que entras a la VM y puedes copiar y pegar comandos.
Navegador (Edge, Chrome, Firefox)	Sí (ya viene)	Abrir SUIF en `http://localhost:8080` y registrar tu llave en GitHub.
Visual Studio Code + extensión *Remote - SSH*	Recomendada	Editar el código que vive dentro de la VM como si estuviera en tu PC. Paso 5.

Lo que no necesitas

XAMPP, Docker, WSL, PHP o PostgreSQL instalados en Windows. Todo el stack vive dentro de la VM. Si tienes Docker Desktop o WSL2, lee el aviso del paso 3: comparten el hipervisor de Windows y pueden impedir que arranque AlmaLinux.

DATOS REALES: NUNCA

La VM es para desarrollo. No cargues respaldos de producción, CURP ni documentos de personas reales. Todo lo que captures debe ser ficticio.

2. Cómo queda el entorno

Tu Windows sólo ejecuta VirtualBox, el navegador y la terminal. SUIF y su base de datos corren dentro de la VM. VirtualBox reenvía dos puertos de tu PC hacia la VM.

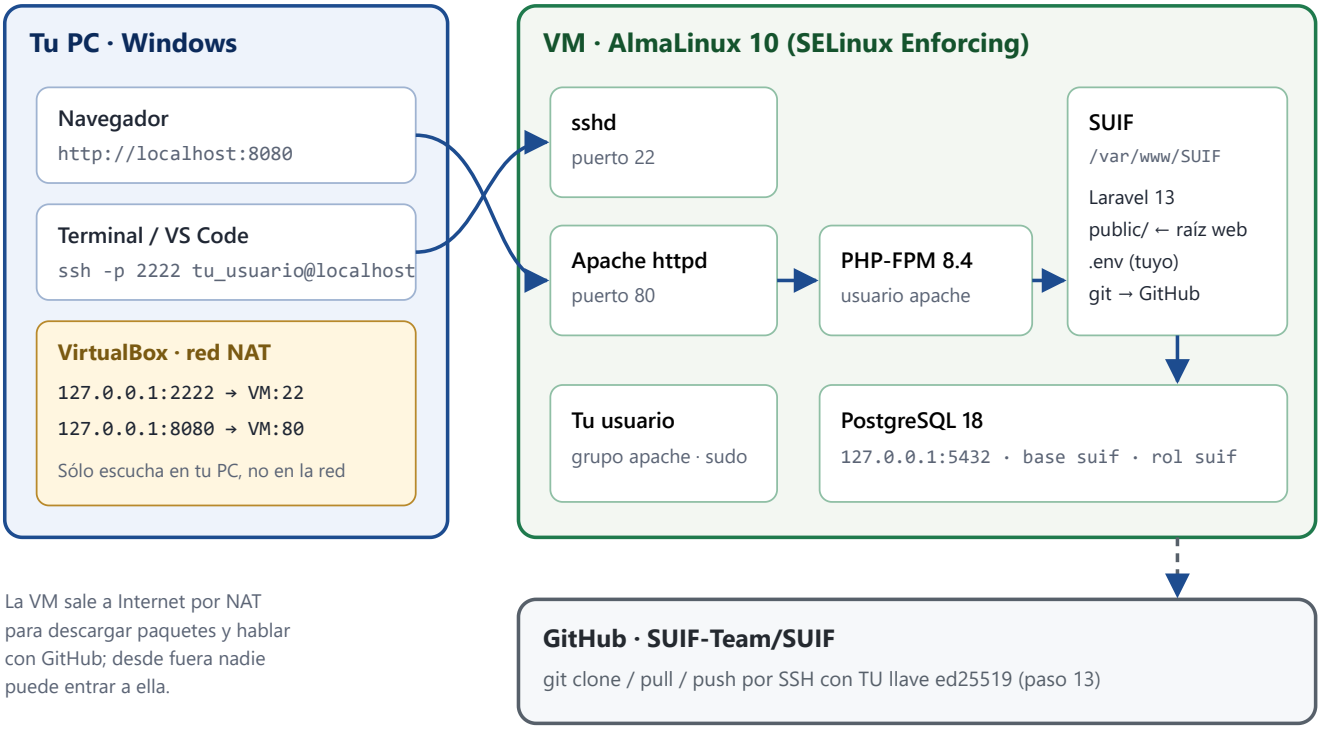


Figura 1. Arquitectura del entorno de desarrollo.

Mapa del recorrido

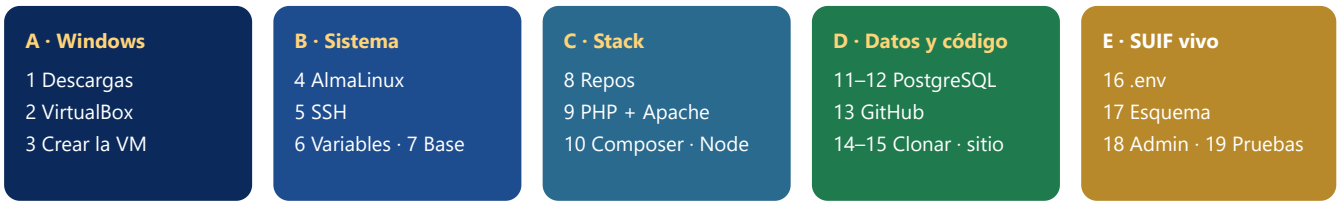


Figura 2. Fases de la instalación. Tiempo aproximado total: 1.5 a 2.5 horas.

3. Tus datos personales

La guía no trae nombres ni correos de nadie. Cada persona define sus propios valores **una sola vez** en el paso 6, en un archivo de variables, y todos los comandos posteriores los toman de ahí.

Variables que vas a llenar (archivo `~/suif-variables.sh`)

Variable	Qué es	Ejemplo (cámbialo)
`GIT_NOMBRE`	Tu nombre como aparecerá en los commits.	Ana Ruiz Soto
`GIT_CORREO`	El correo de tu cuenta de GitHub, o el *noreply* que GitHub te asigna en *Settings → Emails*.	ana.ruiz@ejemplo.com
`GITHUB_USUARIO`	Tu nombre de usuario en GitHub (sin @).	anaruiz
`SUIF_REPO`	Dirección SSH del repositorio. Sólo cámbiala si trabajas sobre un fork.	git@github.com:SUIF-Team/SUIF.git
`SUIF_RAMA`	Rama con la que empiezas.	main
`SUIF_DIR`	Carpeta del proyecto dentro de la VM. No la cambies.	/var/www/SUIF
`SUIF_URL`	Dirección con la que abres SUIF desde Windows. Si cambias el puerto 8080 en el paso 3, cámbialo aquí también.	http://localhost:8080

Datos que defines pero no se guardan en el archivo

Dato	Dónde se pide	Recomendación
Usuario de Linux y su contraseña	Paso 4 (instalador)	Minúsculas y sin espacios, p. ej. `ana`. Es el usuario con el que trabajas siempre.
Contraseña de root	Paso 4	Distinta de la de tu usuario. Casi no la usarás: se trabaja con `sudo`.
Contraseña del rol `suif` de PostgreSQL	Paso 12	**No la inventes:** la guía la genera al azar y la guarda en `~/.pgpass` y en `.env`.
Frase de paso de tu llave SSH	Paso 13	Opcional. Si la pones, git la pedirá en cada `pull` / `push`.
CURP y clave del administrador de prueba	Paso 18	Una CURP **ficticia** de 18 caracteres y una clave de al menos 8.

> **ANÓTALOS EN UN LUGAR SEGURO**
> Usa un gestor de contraseñas. No los pegues en el chat del equipo ni en un issue, y no los subas al repositorio.

1 Descargas

Todo se descarga en Windows. Los paquetes de Linux (PHP, PostgreSQL, etc.) se bajan después, desde la VM.

Qué	Enlace	Nota
VirtualBox 7.2.20 para Windows	Página: virtualbox.org/wiki/Downloads → *Windows hosts* Directo: download.virtualbox.org/virtualbox/7.2.20/VirtualBox-7.2.20-175154-Win.exe	~170 MB. El *Extension Pack* **no** hace falta.
AlmaLinux 10.2 mínimo (x86_64)	Página: almalinux.org/es/get-almalinux → *AlmaLinux OS 10 · x86_64 → ISO Images* → archivo *minimal* Directo: repo.almalinux.org/almalinux/10/isos/x86_64/AlmaLinux-10.2-x86_64-minimal.iso	~2 GB. Es la ISO principal.
AlmaLinux 10.2 mínimo **x86_64_v2**	repo.almalinux.org/almalinux/10/isos/x86_64_v2/AlmaLinux-10.2-x86_64_v2-minimal.iso	**Sólo** si la ISO principal no arranca (Anexo A.1).
Visual C++ Redistributable x64	aka.ms/vs/17/release/vc_redist.x64.exe	Sólo si el instalador de VirtualBox lo pide.
Visual Studio Code	code.visualstudio.com + extensión *Remote - SSH* (Microsoft)	Recomendado.

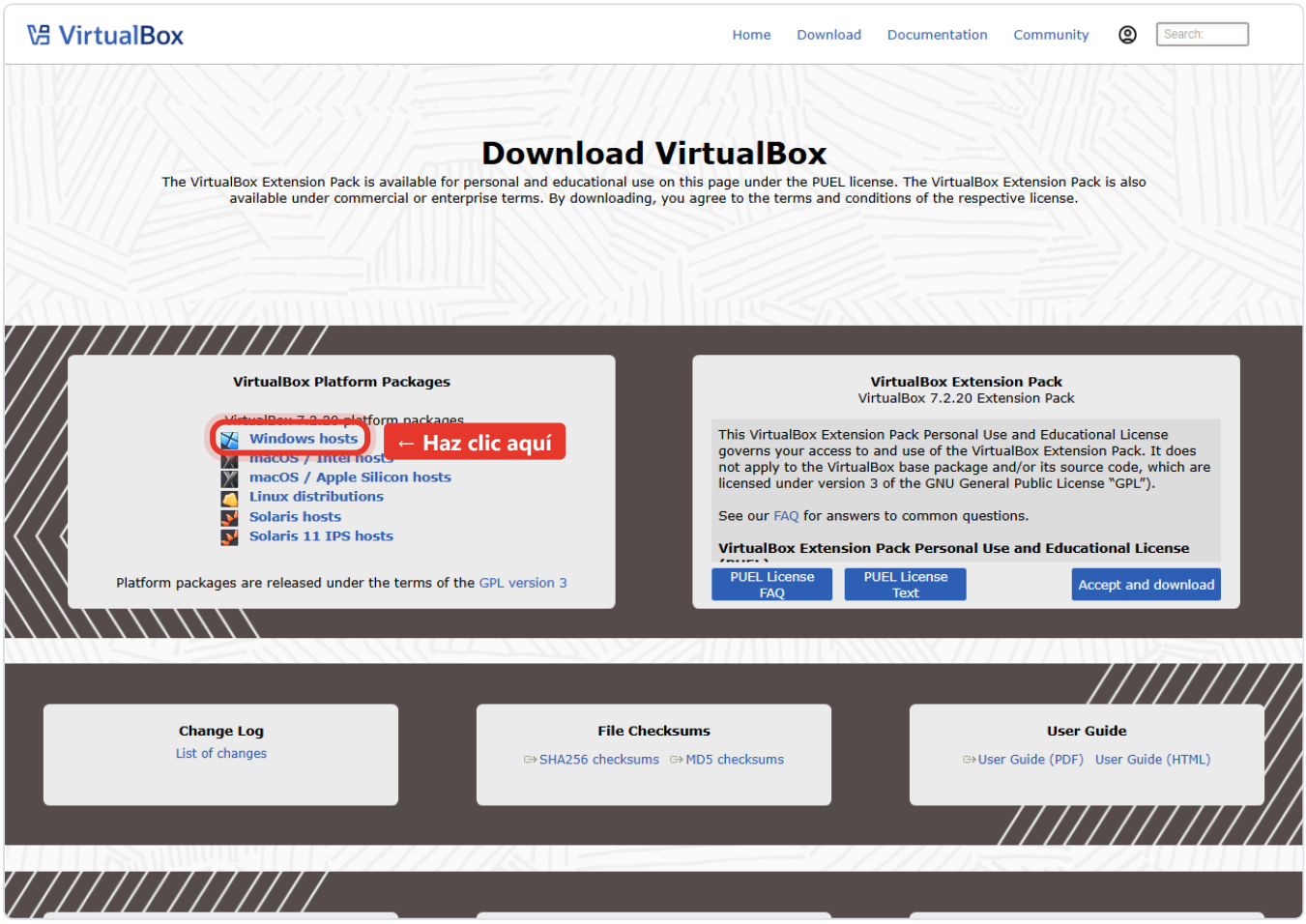


Figura 3. virtualbox.org/wiki/Downloads — en *VirtualBox Platform Packages* elige **Windows hosts**. No es necesario el Extension Pack de la derecha.



Figura 4. almalinux.org/es/get-almalinux — deja seleccionado **AlmaLinux OS 10 · x86_64**, entra a **ISO Images** y descarga el archivo que termina en **-minimal.iso** (no *dvd* ni *boot*).

Comprueba que la ISO no se dañó (opcional, recomendado)

Compara la suma SHA-256 con la publicada en repo.almalinux.org/almalinux/10/isos/x86_64/CHECKSUM:

WINDOWS · POWERSHELL

```
Get-FileHash "$env:USERPROFILE\Downloads\AlmaLinux-10.2-x86_64-minimal.iso" -Algorithm SHA256
```

RESULTADO ESPERADO

El *Hash* que imprime PowerShell es idéntico (sin importar mayúsculas) al renglón `AlmaLinux-10.2-x86_64-minimal.iso` del archivo CHECKSUM.

2 Instalar VirtualBox

1. Ejecuta `VirtualBox-7.2.20-…-Win.exe` con doble clic y acepta el aviso de control de cuentas.
2. Deja las opciones predeterminadas. Si aparece *Missing Dependencies: Python Core / win32api*, contesta **Sí**: sólo afecta a una API que no usamos.
3. Si el instalador avisa que falta **Microsoft Visual C++ Redistributable**, cancela, instala `vc_redist.x64.exe` del paso 1 y vuelve a empezar.
4. Acepta la advertencia de que la red se desconectará unos segundos; es el adaptador virtual que se instala.
5. Al terminar, abre *Oracle VirtualBox*.

Revisa el hipervisor de Windows antes de seguir

AlmaLinux 10 exige un procesador con instrucciones **x86-64-v3**. Cuando Windows tiene activo Hyper-V, que también usan WSL2, Docker Desktop y la *Integridad de memoria*, VirtualBox corre encima de él y a veces no expone esas instrucciones a la VM. El síntoma es que la VM se reinicia sola, se congela o muestra un *kernel panic* al arrancar el instalador.

WINDOWS · POWERSHELL

```
## PowerShell como administrador: ¿está activo el hipervisor de Windows?
bcdedit /enum "{current}" | Select-String hypervisorlaunchtype
Get-CimInstance Win32_ComputerSystem | Select-Object HypervisorPresent
```

RESULTADO IDEAL

`hypervisorlaunchtype Off` (o sin renglón) y `HypervisorPresent False`. Si sale `Auto` / `True`, puedes seguir de todos modos. Sólo si en el paso 4 la VM no arranca, aplica el Anexo A.1.

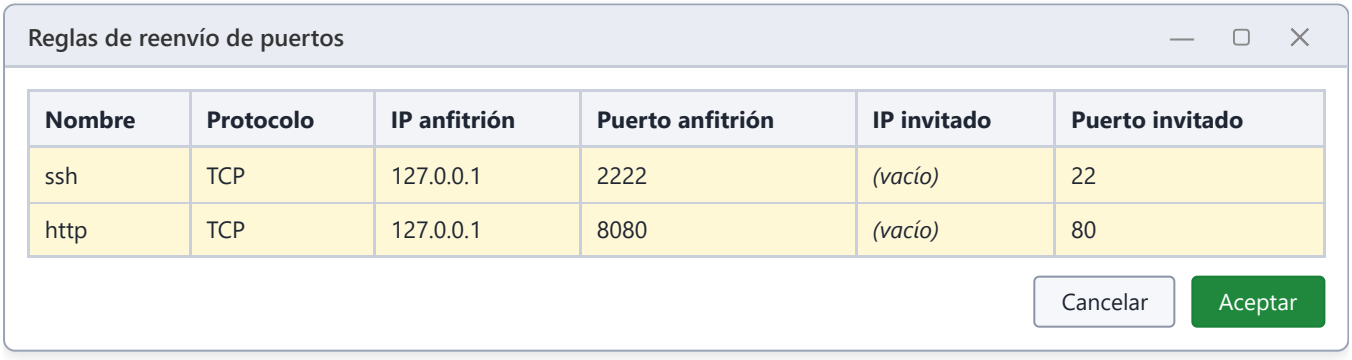
3 Crear la máquina virtual

En VirtualBox: **Máquina** → **Nueva…** (en inglés: *Machine* → *New…*). Llena el asistente así:

Campo	Valor	Por qué
Nombre	`SUIF`	Los comandos de esta guía usan ese nombre.
Imagen ISO	AlmaLinux-10.2-x86_64-minimal.iso	La que descargaste.
Tipo / Subtipo / Versión	Linux / Red Hat / Red Hat (64-bit)	AlmaLinux es compatible con RHEL.
Omitir instalación desatendida *(Skip Unattended Installation)*	**Marcado ✓**	La instalación automática crea un usuario *vboxuser* y no deja elegir la red. Lo haremos a mano.
Memoria base	4096 MB	Suficiente para Apache, PHP y PostgreSQL.
Procesadores	2	Más acelera `composer install` y las pruebas.
EFI	Sin marcar (predeterminado)	No hace falta; cambiarlo complica el arranque.
Disco duro virtual	Crear nuevo · VDI · 40 GB · tamaño dinámico	Ocupa en tu disco sólo lo que se usa.

Reenvío de puertos (lo más importante de este paso)

Con la VM **apagada**: selecciónala → **Configuración** → **Red** → **Adaptador 1**. Deja *Conectado a:* ***NAT***, despliega **Avanzadas** y pulsa **Reenvío de puertos**. Agrega dos reglas con el botón **+**:



Esquema ilustrativo: la ventana real puede verse ligeramente distinta según el idioma y la versión.

Si no encuentras la ventana, la misma configuración se hace desde PowerShell (con VirtualBox cerrado o la VM apagada):

WINDOWS · POWERSHELL

```
$vbox = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $vbox modifyvm "SUIF" --natpf1 "ssh,tcp,127.0.0.1,2222,,22"
& $vbox modifyvm "SUIF" --natpf1 "http,tcp,127.0.0.1,8080,,80"
& $vbox showvminfo "SUIF" --machinereadable | Select-String Forwarding
```

RESULTADO ESPERADO

Dos renglones `Forwarding(0)="http,tcp,127.0.0.1,8080,,80"` y `Forwarding(1)="ssh,tcp,127.0.0.1,2222,,22"`.

¿POR QUÉ 127.0.0.1?

Así la VM sólo es accesible desde tu propia PC y nadie de la red del café o de la Facultad puede entrar a tu SUIF de pruebas. Si el puerto 8080 ya lo usa otro programa, elige otro (p. ej. 8081) y cámbialo también en `SUIF_URL`.

4 Instalar AlmaLinux 10

1. Selecciona la VM **SUIF** y pulsa **Iniciar**.
2. En el menú de arranque sube con la flecha a **Install AlmaLinux 10.2** y pulsa Enter. La opción marcada por defecto, *Test this media & install*, primero revisa la ISO y tarda varios minutos más.
3. Idioma: **Español** → **Español (México)** → Continuar.
4. En el **Resumen de la instalación** configura cada recuadro marcado en rojo:



Esquema ilustrativo del menú principal del instalador (Anaconda).

Recuadro	Qué hacer
Teclado	Agrega **Español (Latinoamérica)** y súbelo al primer lugar si tu teclado tiene Ñ y el **@** en *AltGr+Q*.
Hora y fecha	Región **América**, ciudad **Ciudad de México**. Hora de red encendida.
Destino de la instalación	Selecciona el disco de 40 GB (queda con ✓) y deja *Configuración de almacenamiento: Automática*. Pulsa **Hecho**.
Red y nombre de equipo	Enciende el interruptor de la tarjeta Ethernet (`enp0s3`). **En AlmaLinux 10 viene apagado**: si lo olvidas, la VM no tendrá Internet. Abajo escribe `suif-vm` y pulsa **Aplicar**.
Contraseña de root	Escribe una contraseña. **No** marques *Permitir inicio de sesión de root con contraseña por SSH*.
Creación de usuario	Nombre completo y nombre de usuario (p. ej. `ana`). Marca **Hacer administrador a este usuario**: así entra al grupo *wheel* y puede usar `sudo` . Define la contraseña.

5. Pulsa **Empezar la instalación**. Tarda de 5 a 15 minutos.
6. Pulsa **Reiniciar el sistema**. Si vuelve a aparecer el menú del instalador, apaga la VM, quita la ISO en *Configuración → Almacenamiento* y vuelve a iniciar.
7. Inicia sesión en la consola con tu usuario y comprueba la red:

VM · TERMINAL SSH

```
ip -br address
ping -c 3 github.com
cat /etc/almalinux-release
```

RESULTADO ESPERADO

`enp0s3 UP 10.0.2.15/24` , tres respuestas de github.com y `AlmaLinux release 10.2 (…)` . Si el ping falla, revisa el recuadro «Red» (Anexo A.3).

5 Entrar por SSH desde Windows

A partir de aquí ya no escribas en la ventana de VirtualBox: no permite copiar y pegar. Abre la **Terminal de Windows** (PowerShell) y conéctate a la VM por el puerto 2222.

WINDOWS · POWERSHELL

```
ssh -p 2222 tu_usuario_linux@localhost
```

La primera vez pregunta *Are you sure you want to continue connecting (yes/no)?*. Escribe `yes` y luego tu contraseña de Linux. Para pegar en la terminal usa **clic derecho** o **Ctrl+V**.

Atajo: un nombre corto para la VM

Agrega este bloque al archivo `C:\Users\TU_USUARIO_WINDOWS\.ssh\config` (créalo si no existe; sin extensión):

WINDOWS · POWERSHELL

```
## Abre (o crea) el archivo en el Bloc de notas
New-Item -ItemType Directory -Force "$env:USERPROFILE\.ssh" | Out-Null
notepad "$env:USERPROFILE\.ssh\config"

## Pega esto, cambia el usuario, guarda y cierra:
Host suif-vm
    HostName 127.0.0.1
    Port 2222
    User tu_usuario_linux
```

Desde ese momento basta con `ssh suif-vm`.

Visual Studio Code dentro de la VM (recomendado)

1. Instala la extensión **Remote - SSH** de Microsoft.
2. Pulsa **F1** → *Remote-SSH: Connect to Host…* → **suif-vm** → plataforma **Linux**.
3. Cuando termines el paso 14, abre la carpeta `/var/www/SUIF`. Lo que edites se guarda directamente en la VM y Apache lo sirve al instante.

> **NO EDITES EL CÓDIGO DESDE WINDOWS**
>
> No copies el proyecto a una carpeta de Windows ni a OneDrive, y no lo compartas con «carpetas compartidas» de VirtualBox: cambian los finales de línea y los permisos, y SELinux bloquea esa carpeta. El código vive en la VM.

6 Cargar tus variables

Crea el archivo con tus datos. Todo lo que sigue es **VM · terminal SSH**.

VM · TERMINAL SSH

```
cat > ~/suif-variables.sh <<'EOF'
# ── Datos personales de SUIF · edítalos una sola vez ──
export GIT_NOMBRE="TU_NOMBRE_COMPLETO"
export GIT_CORREO="TU_CORREO_DE_GITHUB"
export GITHUB_USUARIO="TU_USUARIO_DE_GITHUB"

# ── Del proyecto · normalmente no se cambian ──
export SUIF_REPO="git@github.com:SUIF-Team/SUIF.git"
export SUIF_RAMA="main"
export SUIF_DIR="/var/www/SUIF"
export SUIF_URL="http://localhost:8080"
EOF
nano ~/suif-variables.sh
```

En `nano` cambia los tres valores **«TU_...»**, conservando las comillas. Guarda con **Ctrl+O** y Enter, y sal con **Ctrl+X**. Después haz que se carguen en cada sesión:

VM · TERMINAL SSH

```
echo 'source ~/suif-variables.sh' >> ~/.bashrc
source ~/suif-variables.sh

# Comprobación: no debe quedar ningún «TU_»
for v in GIT_NOMBRE GIT_CORREO GITHUB_USUARIO SUIF_REPO SUIF_RAMA SUIF_DIR SUIF_URL; do
  case "${!v}" in *TU_*|"") echo "FALTA: $v";; *) echo "ok  $v=${!v}";; esac
done
```

RESULTADO ESPERADO

Siete renglones que empiezan con `ok`. Si alguno dice `FALTA`, vuelve a `nano ~/suif-variables.sh`.

7 Sistema base y cortafuegos

VM · TERMINAL SSH

```
sudo dnf upgrade --refresh -y
sudo dnf install -y git curl wget unzip tar nano vim-enhanced bash-completion \
  dnf-plugins-core policycoreutils-python-utils firewalld
sudo timedatectl set-timezone America/Mexico_City

sudo systemctl enable --now firewalld
sudo firewall-cmd --permanent --add-service=ssh
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
```

Si `dnf upgrade` instaló un kernel nuevo, reinicia y vuelve a entrar por SSH:

VM · TERMINAL SSH

```
sudo reboot
```

Comprobación del paso 7

VM · TERMINAL SSH

```
systemctl is-active sshd firewalld
firewall-cmd --list-services
getenforce
timedatectl | grep "Time zone"
```

RESULTADO ESPERADO

`active` dos veces; la lista incluye `http` y `ssh`; `Enforcing`; `America/Mexico_City`.

NO DESACTIVES SELINUX

El server de despliegue lo tiene en *Enforcing*. Si en tu VM lo apagas, tu código funcionará aquí y fallará allá. El paso 14 pone las etiquetas correctas.

8 Repositorios de paquetes

AlmaLinux 10 trae PHP 8.3 y PostgreSQL 16. SUIF usa PHP 8.4 y PostgreSQL 18, así que se agregan Remi (PHP) y PGDG (PostgreSQL), más CRB y EPEL, de los que depende Remi. Los comandos son los que publican el asistente oficial de Remi y la página oficial de PostgreSQL para EL 10.

VM · TERMINAL SSH

```
sudo dnf config-manager --set-enabled crb
sudo dnf install -y https://dl.fedoraproject.org/pub/epel/epel-release-latest-10.noarch.rpm
sudo dnf install -y https://rpms.remirepo.net/enterprise/remi-release-10.rpm
sudo dnf install -y https://download.postgresql.org/pub/repos/yum/reporpms/EL-10-x86_64/pgdg-redhat-repo-latest.noarch.rpm
```

NO EJECUTES «DNF MODULE DISABLE POSTGRESQL»

La guía anterior lo pedía. En AlmaLinux 10 no existe ese módulo: el comando falla con *Problems in request: missing groups or modules: postgresql* y confunde a quien lo ve. PostgreSQL sólo lo pide en EL 8.

Comprobación del paso 8

VM · TERMINAL SSH

```
dnf repolist --enabled | grep -Ei "crb|epel|remi|pgdg18"
```

RESULTADO ESPERADO

Aparecen `crb`, `epel`, `remi-modular`, `remi-safe` y `pgdg18`. Si dnf pregunta si importa una llave GPG, contesta `y`.

9 PHP 8.4 y Apache

Remi publica PHP 8.4 como flujo de módulo en su repositorio `remi-modular`. En RHEL 10 los módulos ya no vienen en el sistema, pero `dnf` los sigue soportando.

VM · TERMINAL SSH

```
sudo dnf module install -y php:remi-8.4
sudo dnf install -y httpd \
  php-cli php-fpm php-common php-mbstring php-xml php-pdo php-pgsql \
  php-process php-zip php-intl php-gd php-bcmath php-opcache php-sodium
```

Ajustes de PHP para SUIF

VM · TERMINAL SSH

```
sudo tee /etc/php.d/99-suif.ini > /dev/null <<'EOF'
memory_limit = 512M
upload_max_filesize = 20M
post_max_size = 25M
max_execution_time = 60
date.timezone = America/Mexico_City
EOF
```

Que PHP-FPM cree archivos que tu usuario pueda reescribir

Laravel escribe cachés a veces como tu usuario (`php artisan`) y a veces como `apache` (el navegador). Con esta máscara los archivos nacen con escritura para el grupo y no aparece el clásico *Permission denied* en `storage/` .

VM · TERMINAL SSH

```
sudo mkdir -p /etc/systemd/system/php-fpm.service.d
printf '[Service]\nUMask=0002\n' | sudo tee /etc/systemd/system/php-fpm.service.d/umask.conf
sudo systemctl daemon-reload

sudo mv /etc/httpd/conf.d/welcome.conf /etc/httpd/conf.d/welcome.conf.disabled
sudo systemctl enable --now php-fpm httpd
```

Comprobación del paso 9

VM · TERMINAL SSH

```
php -v | head -1
php -m | grep -Ei "^(pdo_pgsql|pdo_sqlite|mbstring|intl|gd|bcmath|zip|sodium|Zend OPcache)$"
httpd -M 2>/dev/null | grep -E "rewrite|proxy_fcgi"
systemctl is-active php-fpm httpd
```

RESULTADO ESPERADO

`PHP 8.4.x` (cualquier parche de la rama 8.4 sirve); las nueve extensiones; `rewrite_module` y `proxy_fcgi_module` ; `active` dos veces. `pdo_sqlite` es indispensable: las pruebas automáticas corren sobre SQLite en memoria.

10 Composer y Node.js

Composer 2

Instalador oficial de getcomposer.org, verificando su firma antes de ejecutarlo:

VM · TERMINAL SSH

```
cd /tmp
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
ESPERADO="$(php -r "copy('https://composer.github.io/installer.sig', 'php://stdout');")"
OBTENIDO="$(php -r "echo hash_file('sha384', 'composer-setup.php');")"
if [ "$ESPERADO" = "$OBTENIDO" ]; then
  sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer
else
  echo "ERROR: el instalador de Composer NO pasó la verificación"
fi
rm -f composer-setup.php
composer --version
```

RESULTADO ESPERADO

`Composer version 2.x.x` . Si sale el mensaje de ERROR, no continúes: repite el bloque; casi siempre es una descarga cortada.

Node.js 24 LTS (opcional hoy)

SUIF todavía no tiene paso de compilación: los estilos y scripts se sirven tal cual desde `public/assets` . Node.js forma parte del stack acordado y conviene tenerlo, pero si tienes prisa puedes saltar este bloque.

VM · TERMINAL SSH

```
cd /tmp
curl -fsSLO https://nodejs.org/dist/latest-v24.x/SHASUMS256.txt
NODE_TAR="$(grep -o 'node-v24\.[0-9.]*-linux-x64\.tar\.xz' SHASUMS256.txt | head -1)"
curl -fsSLO "https://nodejs.org/dist/latest-v24.x/$NODE_TAR"
sha256sum -c SHASUMS256.txt --ignore-missing
sudo tar -xJf "$NODE_TAR" -C /usr/local --strip-components=1 \
  --exclude=CHANGELOG.md --exclude=LICENSE --exclude=README.md
rm -f "$NODE_TAR" SHASUMS256.txt
node -v; npm -v
```

RESULTADO ESPERADO

`sha256sum` responde `…linux-x64.tar.xz: OK` y `node -v` devuelve `v24.x.x` . Si dice `FAILED` , borra el archivo y repite.

11 PostgreSQL 18

VM · TERMINAL SSH

```
sudo dnf install -y postgresql18-server postgresql18-contrib
sudo /usr/pgsql-18/bin/postgresql-18-setup initdb
sudo systemctl enable --now postgresql-18
```

INITDB SE EJECUTA UNA SOLA VEZ

Si lo repites, responde que el directorio de datos ya existe. No es un error que tengas que «arreglar» borrando nada; sigue adelante.

Autenticación por contraseña en 127.0.0.1

Laravel y tú se conectan por TCP a `127.0.0.1` con usuario y contraseña. Las líneas `host` de `pg_hba.conf` deben decir `scram-sha-256`, que es lo que deja el instalador actual. Revísalo y, si dicen `ident`, este bloque las corrige (guarda antes una copia):

VM · TERMINAL SSH

```
PGHBA=/var/lib/pgsql/18/data/pg_hba.conf
sudo grep -nE '^(local|host)' "$PGHBA"

sudo cp -n "$PGHBA" "$PGHBA.original"
sudo sed -i -E '/^host\s+all\s+all\s+(127\.0\.0\.1\/32|::1\/128)/ s/\S+\s*$/scram-sha-256/'
"$PGHBA"
sudo systemctl reload postgresql-18
sudo grep -nE '^host' "$PGHBA"
```

✕ Sin -h → socket local → «peer»

`psql -U suif -d suif`

PostgreSQL compara tu usuario de Linux (ana) con el rol (suif) y rechaza:

`Peer authentication failed`

✓ Con -h 127.0.0.1 → TCP → scram

`psql -h 127.0.0.1 -U suif -d suif`

Usa la contraseña de ~/.pgpass (paso 12), igual que Laravel. Los objetos que crees quedan a nombre de suif, como debe ser.

Figura 5. La causa número uno de «PostgreSQL no me deja entrar». En esta guía **siempre** se usa `-h 127.0.0.1`.

Comprobación del paso 11

VM · TERMINAL SSH

```
systemctl is-active postgresql-18
psql --version
sudo grep -cE '^host\s+all\s+all\s+(127\.0\.0\.1/32|::1/128)\s+scram-sha-256'
/var/lib/pgsql/18/data/pg_hba.conf
```

RESULTADO ESPERADO

`active`, `psql (PostgreSQL) 18.x` y el número `2`.

12 Rol, base de datos y .pgpass

Se crea el rol `suif` (sin privilegios de superusuario) dueño de la base `suif`. La contraseña se genera al azar, sólo con letras y números para que no rompa `.pgpass` ni `.env`, y nunca se escribe a mano ni queda en el historial.

VM · TERMINAL SSH

```
cd /tmp   # el usuario postgres no puede entrar a tu carpeta personal
SUIF_BD_CLAVE="$(tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 24)"

sudo -u postgres psql -v ON_ERROR_STOP=1 -v clave="$SUIF_BD_CLAVE" <<'SQL'
CREATE ROLE suif LOGIN PASSWORD :'clave';
CREATE DATABASE suif OWNER suif ENCODING 'UTF8' TEMPLATE template0;
SQL

printf '127.0.0.1:5432:*:suif:%s\n' "$SUIF_BD_CLAVE" > ~/.pgpass
chmod 600 ~/.pgpass
unset SUIF_BD_CLAVE
cd ~
```

¿QUÉ ES ~/.PGPASS?

Un archivo que sólo tú puedes leer, donde `psql` busca la contraseña. Gracias a él ningún comando de esta guía te pide teclearla. Si el permiso no es exactamente `600`, psql lo ignora en silencio y vuelve a preguntar.

Comprobación del paso 12

VM · TERMINAL SSH

```
psql -h 127.0.0.1 -U suif -d suif -c "SELECT current_user, current_database();"
```

RESULTADO ESPERADO

Sin pedir contraseña, una fila con `suif | suif`. Si pide contraseña o dice *password authentication failed*, ve al Anexo A.9.

SI EL CREATE ROLE DICE «ALREADY EXISTS»

Ya habías corrido este paso. Para no quedarte con una contraseña desconocida, cámbiala y regenera `.pgpass` con el bloque del Anexo A.9.

13 Git y tu llave SSH de GitHub

Cada persona usa **su propia llave**, creada dentro de su VM. Nunca copies la llave privada de otra persona ni compartas la tuya.

13.1 Identidad de git y llave nueva

VM · TERMINAL SSH

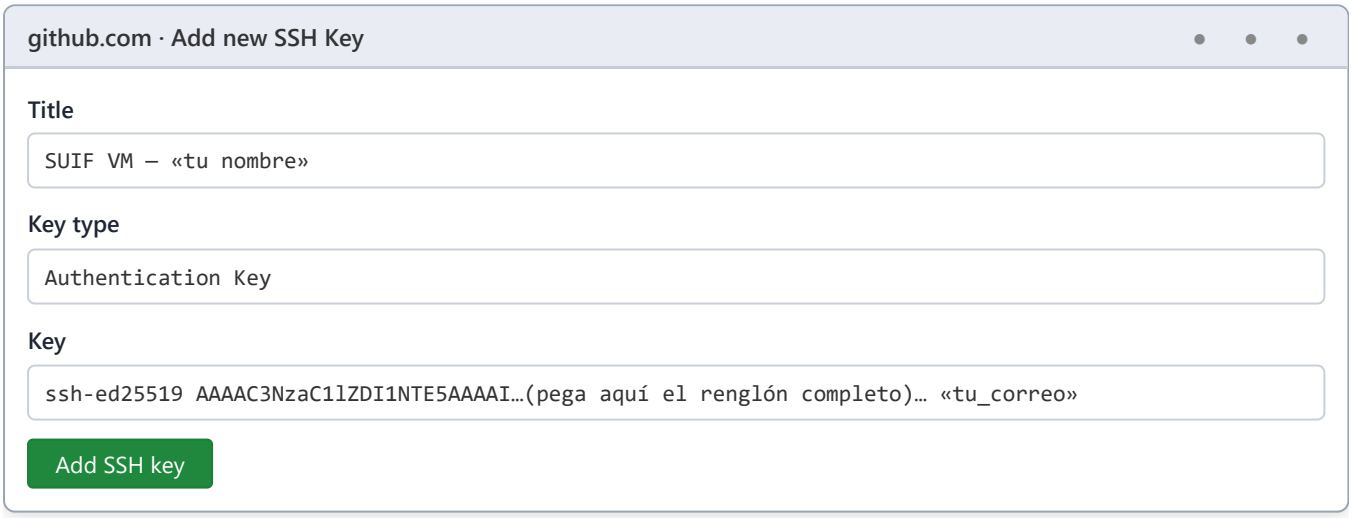
```
git config --global user.name  "$GIT_NOMBRE"
git config --global user.email "$GIT_CORREO"
git config --global init.defaultBranch main

ssh-keygen -t ed25519 -C "$GIT_CORREO" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

`ssh-keygen` te pide una frase de paso. Pulsa Enter dos veces si no quieres ninguna. El último comando imprime un renglón que empieza con `ssh-ed25519 AAAA…` y termina con tu correo: **selecciónalo completo y cópialo**.

13.2 Registra la llave en tu cuenta de GitHub

En el navegador de Windows: tu foto (arriba a la derecha) → **Settings** → **SSH and GPG keys** → **New SSH key**, o directo en github.com/settings/ssh/new.



Esquema ilustrativo del formulario de GitHub. GitHub puede pedirte confirmar con tu contraseña o con 2FA.

13.3 Prueba la conexión

VM · TERMINAL SSH

```
ssh -T git@github.com
```

La primera vez pregunta si confías en el servidor. Antes de escribir `yes`, comprueba que la huella sea exactamente la que GitHub publica para ED25519:

```
SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU
```

RESULTADO ESPERADO

Hi «tu_usuario»! You've successfully authenticated, but GitHub does not provide shell access. Es normal que el comando termine con código 1. Para verificarlo con tu variable:

VM · TERMINAL SSH

```
ssh -T git@github.com 2>&1 | grep -q "Hi $GITHUB_USUARIO!" && echo "GitHub OK" || echo "Revisa el Anexo A.4"
```

14 Clonar SUIF, permisos y SELinux

14.1 Tu usuario entra al grupo apache

VM · TERMINAL SSH

```
sudo usermod -aG apache "$USER"
echo 'umask 0002' >> ~/.bashrc
exit
```

> **VUELVE A ENTRAR**
>
> El grupo nuevo sólo aplica en una sesión nueva. Conéctate otra vez con `ssh suif-vm` y comprueba que `id` muestre `apache` entre tus grupos y que `umask` responda `0002`.

14.2 Clonar

VM · TERMINAL SSH

```
sudo install -d -o "$USER" -g apache -m 2775 "$SUIF_DIR"
git clone "$SUIF_REPO" "$SUIF_DIR"
cd "$SUIF_DIR"
git switch "$SUIF_RAMA"
git log --oneline -1
```

14.3 Etiquetas de SELinux

SELinux decide por etiqueta qué puede leer y escribir Apache: todo el proyecto se puede leer, y sólo `storage/` y `bootstrap/cache/` se pueden escribir.

VM · TERMINAL SSH

```
sudo semanage fcontext -a -t httpd_sys_content_t    "$SUIF_DIR(/.*)?"
sudo semanage fcontext -a -t httpd_sys_rw_content_t "$SUIF_DIR/storage(/.*)?"
sudo semanage fcontext -a -t httpd_sys_rw_content_t "$SUIF_DIR/bootstrap/cache(/.*)?"
sudo restorecon -R "$SUIF_DIR"
sudo setsebool -P httpd_can_network_connect_db on

chmod -R g+w "$SUIF_DIR/storage" "$SUIF_DIR/bootstrap/cache"
```

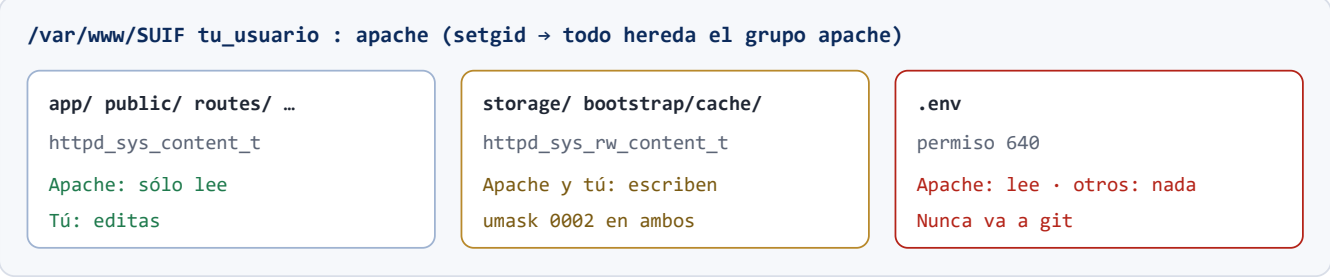


Figura 6. Quién puede qué dentro del proyecto.

Comprobación del paso 14

VM · TERMINAL SSH

```
ls -Zd "$SUIF_DIR" "$SUIF_DIR/storage" "$SUIF_DIR/bootstrap/cache"
getsebool httpd_can_network_connect_db
git -C "$SUIF_DIR" remote -v
```

RESULTADO ESPERADO

La raíz con `httpd_sys_content_t`, las otras dos con `httpd_sys_rw_content_t`, el booleano en `on` y el remoto `origin git@github.com:SUIF-Team/SUIF.git`.

15 Sitio de Apache

Apache debe servir `public/`, nunca la raíz del proyecto: de lo contrario el `.env` quedaría visible desde el navegador.

VM · TERMINAL SSH

```
sudo tee /etc/httpd/conf.d/suif.conf > /dev/null <<EOF
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot $SUIF_DIR/public
    DirectoryIndex index.php

    <Directory $SUIF_DIR/public>
        Options FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog  /var/log/httpd/suif-error.log
    CustomLog /var/log/httpd/suif-access.log combined
</VirtualHost>
EOF
sudo apachectl configtest && sudo systemctl reload httpd
grep -E "DocumentRoot|Directory " /etc/httpd/conf.d/suif.conf
```

RESULTADO ESPERADO

`Syntax OK` y las rutas ya con `/var/www/SUIF/public` escrito. Si ves `$SUIF_DIR` literal, no cargaste tus variables: `source ~/suif-variables.sh` y repite.

POR QUÉ ALLOWOVERRIDE ALL

El archivo `public/.htaccess` de Laravel envía todas las URL a `index.php`. Sin esa línea, cualquier página distinta de la portada responde 404.

16 Archivo .env y dependencias

El `.env` es tuyo: guarda la configuración de tu VM y **no se sube al repositorio**. Se parte de `.env.example`, que ya trae `DB_HOST=127.0.0.1`, `MAIL_MAILER=log` y el modo de desarrollo.

VM · TERMINAL SSH

```
cd "$SUIF_DIR"
cp -n .env.example .env
sed -i -e "s|^APP_URL=.*|APP_URL=$SUIF_URL|" \
       -e "s|^DB_PASSWORD=.*|DB_PASSWORD=$(cut -d: -f5 ~/.pgpass)|" .env
chmod 640 .env

composer install --no-interaction --prefer-dist
php artisan key:generate
php artisan storage:link
php artisan optimize:clear

grep -E '^(APP_ENV|APP_DEBUG|APP_URL|DB_CONNECTION|DB_HOST|DB_DATABASE|DB_USERNAME|MAIL_MAILER)='
.env
```

RESULTADO ESPERADO

`APP_ENV=local`, `APP_DEBUG=true`, `APP_URL=http://localhost:8080`, `pgsql`, `127.0.0.1`, `suif`, `suif`, `log`. Composer termina sin errores rojos. Puede tardar unos minutos.

EN TU VM NO SE CACHEA LA CONFIGURACIÓN

No ejecutes `config:cache`, `route:cache` ni `view:cache` en desarrollo. Con la configuración cacheada, `php artisan test` ignora `phpunit.xml` y las pruebas borran tablas de la base real. `tests/TestCase.php` lo detecta y aborta, pero la costumbre es la primera defensa.

17 Esquema de la base de datos

SUIF no usa migraciones de Laravel. El esquema completo, con sus 39 tablas y los catálogos con los que arranca el sistema, vive en un solo archivo: `database/scripts/instalacion.sql`.



Figura 7. Contenido de database/scripts/.

COMPRUEBA QUE TU RAMA TRAIGA INSTALACION.SQL

La reorganización de `database/scripts/` es del 14 de octubre de 2026. Si `ls "$SUIF_DIR/database/scripts"` no muestra `instalacion.sql`, tu rama es anterior: actualízala (`git pull`) o cambia a la rama donde ya está y vuelve a revisar. No uses los archivos `suif_…` de una rama vieja.

VM · TERMINAL SSH

```
cd "$SUIF_DIR/database/scripts"
psql -h 127.0.0.1 -U suif -d suif -v ON_ERROR_STOP=1 --single-transaction -f instalacion.sql
cd "$SUIF_DIR"
```

RESULTADO ESPERADO

Un renglón de conteos al final: `tablas 39`, `roles 4`, `privilegios 7`, `estados_convocatoria 1`, `regimenes 11`, `formas_pago 4`, `bancos 17`. Si aparece cualquier `ERROR`, la transacción se revierte completa y la base queda vacía otra vez: corrige la causa y repite el mismo comando.

LO QUE NO SE USA PARA INSTALAR

Ni `limpiar_padron.sql` ni `reponer_tablas.sql`, que son de mantenimiento, ni los scripts de `historico/`, que ya están incluidos en `instalacion.sql`. Si en alguna rama vieja ves archivos que empiezan con `suif_…` (`suif_lleno.sql`, `suif_instalacion_completa.sql`…), ignóralos: están desfasados.

Comprobación del paso 17

VM · TERMINAL SSH

```
psql -h 127.0.0.1 -U suif -d suif \
  -c "SELECT count(*) AS tablas FROM information_schema.tables WHERE table_schema = 'public';" \
  -c "SELECT tableowner, count(*) FROM pg_tables WHERE schemaname = 'public' GROUP BY 1;" \
  -c "SELECT rol_id_rol, rol_tipo_rol FROM rol ORDER BY 1;" \
  -c "SELECT count(*) AS metodos_pago FROM metodo_pago;"
```

RESULTADO ESPERADO

`39` tablas; todas a nombre de `suif`; los roles `Persona`, `Superusuario`, `Admin UIF` y `Admin DEC`; y `metodos_pago` mayor que cero.

18 Administrador y datos de prueba

18.1 Tu Superusuario

La primera cuenta administrativa sólo se puede crear desde la terminal. Usa una CURP **ficticia** de 18 caracteres: con ella vas a iniciar sesión. El comando pide la clave sin mostrarla (mínimo 8 caracteres).

VM · TERMINAL SSH

```
cd "$SUIF_DIR"
php artisan suif:crear-admin --curp=PRUE800101HDFXXX01 --nombre=Nombre --paterno=Apellido --
materno=Apellido
```

18.2 Primer ingreso desde Windows

1. Abre **http://localhost:8080/login** en el navegador de Windows.
2. Entra con la CURP y la clave del paso 18.1. Llegas al tablero administrativo (`/admin/dashboard`).

18.3 Datos de prueba, creados desde la aplicación

No hay un script de datos de prueba vigente: se generan usando SUIF tal como lo usaría una persona.

#	Dónde	Qué hacer
1	Admin → Convocatorias	Crea una convocatoria **Vigente** con fechas que incluyan hoy. La que siembran los catálogos puede tener la ventana vencida, y sin una vigente el pre-registro está cerrado.
2	Admin → Sedes	Da de alta una sede con cupo y, dentro de ella, al menos un horario de aplicación.
3	Admin → Responsables de pago	Registra un responsable activo; sin él no se genera el formato de pago.
4	Ventana privada → `/persona/preregistro`	Pre-regístrate como una persona ficticia, con otra CURP inventada. Así recorres el flujo completo: documentos, revisión, pago y sede.
5	`storage/logs/`	Con `MAIL_MAILER=log` los correos no salen a Internet: se escriben en el log del día. Ahí encuentras, por ejemplo, el enlace de recuperación de clave.

VM · TERMINAL SSH

```
## Ver los últimos correos «enviados»
tail -n 80 "$SUIF_DIR/storage/logs/laravel-$(date +%F).log"
```

> **TOMA UNA INSTANTÁNEA AHORA**
>
> Con la VM apagada (`sudo poweroff`), en VirtualBox selecciona **SUIF** → **Instantáneas** → **Tomar** y nómbrala «SUIF instalado». Si algún día rompes el entorno, regresas a este punto en segundos.

19 Verificación final

Recorre la lista completa. Si un renglón no coincide, busca el síntoma en el Anexo A antes de dar por terminada la instalación.

#	Comando (VM)	Resultado esperado
1	`systemctl is-active httpd php-fpm postgresql-18`	`active` tres veces
2	`php -v | head -1`	PHP 8.4.x
3	`composer --version`	Composer 2.x
4	`psql --version`	psql (PostgreSQL) 18.x
5	`node -v` (si lo instalaste)	v24.x
6	`getenforce`	Enforcing
7	`php artisan about --only=environment`	Laravel 13.x, PHP 8.4.x, entorno *local*
8	`php artisan route:list | tail -1`	«Showing [N] routes» sin errores
9	`curl -s -o /dev/null -w "%{http_code}\n" http://localhost/login`	200
10	`ssh -T git@github.com`	Hi «tu_usuario»!
11	`sudo tail -n 5 /var/log/httpd/suif-error.log`	Sin errores recientes
12	Navegador de Windows: `http://localhost:8080`	Portada de SUIF con sus estilos

Las pruebas automáticas

VM · TERMINAL SSH

```
cd "$SUIF_DIR"
php artisan config:clear      # SIEMPRE antes de probar
php artisan test
```

RESULTADO ESPERADO

Las pruebas corren sobre SQLite en memoria y terminan en `Tests: N passed`. Si ves *PRUEBAS ABORTADAS: la conexión no es SQLite en memoria*, había configuración cacheada: repite `config:clear`.

Trabajo diario

Encender y apagar

Inicia la VM desde VirtualBox; si no quieres ver su ventana, usa *Iniciar → Inicio sin interfaz*. Después entra con `ssh suif-vm` o desde VS Code. Para apagarla: `sudo poweroff`. Apache, PHP-FPM y PostgreSQL arrancan solos.

Traer cambios y trabajar en una rama

VM · TERMINAL SSH

```
cd "$SUIF_DIR"
git switch main
git pull --ff-only
git switch -c feat/descripcion-corta      # tu rama de trabajo

composer install --no-interaction --prefer-dist   # si cambió composer.lock
php artisan optimize:clear                        # siempre después de un pull
```

Si el pull trajo un script nuevo en `database/scripts/historico/` (por ejemplo `13_….sql`), córrelo sobre tu base con el mismo comando del paso 17, cambiando el nombre del archivo: los incrementales son idempotentes. **Nunca vuelvas a correr** `instalacion.sql` sobre tu base con datos: es sólo para una base vacía y fallaría porque las tablas ya existen.

Subir tu trabajo

VM · TERMINAL SSH

```
git status
git add archivos
git commit -m "feat: qué cambia"
git push -u origin HEAD
```

Después abre el Pull Request en GitHub contra `main`. Nunca hagas `git add .env`.

> **RAMAS VIEJAS Y EL .ENV**
>
> Las ramas anteriores al 23 de septiembre de 2026 todavía tenían `.env` en git. Si cambias a una de ellas, git puede sobrescribir el tuyo sin avisar. Respáldalo antes con `cp .env ~/env-respaldo` o cambia de rama con `git switch --no-overwrite-ignore «rama»`.

Anexo A — Problemas conocidos

A.1 La VM no arranca, se reinicia sola o muestra kernel panic

Síntomas: pantalla negra o reinicio en bucle al elegir *Install AlmaLinux*; *Fatal glibc error: CPU does not support x86-64-v3*; tortuga verde en la barra de estado de VirtualBox. **Causa:** el hipervisor de Windows no le pasa a la VM las instrucciones x86-64-v3 que exige AlmaLinux 10. **Solución** (elige una):

1. Desactiva el hipervisor de Windows, si no necesitas WSL2 ni Docker Desktop: *Seguridad de Windows → Seguridad del dispositivo → Aislamiento del núcleo → Integridad de memoria: Desactivado*; luego ejecuta en PowerShell como administrador y reinicia Windows:

WINDOWS · POWERSHELL

```
bcdedit /set hypervisorlaunchtype off
Disable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V-All, VirtualMachinePlatform
-NoRestart
## Para revertirlo después: bcdedit /set hypervisorlaunchtype auto
```

2. Si necesitas WSL2 o Docker, usa la ISO **x86_64_v2** del paso 1: borra la VM, créala de nuevo con esa ISO y sigue igual. La guía no cambia.

A.2 «VT-x is not available (VERR_VMX_NO_VMX)» o «AMD-V is disabled»

La virtualización está apagada en el BIOS/UEFI. Entra al BIOS al encender (F2, F10, Supr o Esc, según la marca) y activa *Intel Virtualization Technology* o *SVM Mode*.

A.3 La VM no tiene Internet

En el instalador quedó apagada la tarjeta Ethernet. Arréglalo desde la consola de la VM:

VM · TERMINAL SSH

```
nmcli device status
sudo nmcli connection modify enp0s3 connection.autoconnect yes
sudo nmcli connection up enp0s3
```

A.4 «ssh: connect to host localhost port 2222: Connection refused» / «Permission denied (publickey)»

Connection refused: la VM está apagada o falta la regla de reenvío (paso 3). **Permission denied (publickey)** al hablar con GitHub: la llave no está registrada en tu cuenta, o pegaste la privada en lugar de la `.pub`. Diagnóstico:

VM · TERMINAL SSH

```
ls -l ~/.ssh/
ssh -vT git@github.com 2>&1 | grep -E "Offering|Authenticated|denied"
```

A.5 «ERROR: Permission to SUIF-Team/SUIF.git denied to «usuario»»

Tu llave funciona, pero tu cuenta no es colaboradora del repositorio o no aceptaste la invitación. Revisa github.com/SUIF-Team/SUIF/invitations o pide al responsable que te agregue.

A.6 «No match for argument» / «missing groups or modules: postgresql»

Viene de `dnf module disable postgresql`, que en AlmaLinux 10 no aplica. Ignóralo y sigue con el paso 11.

A.7 «Unable to resolve argument php:remi-8.4» / dnf no encuentra PHP 8.4

VM · TERMINAL SSH

```
sudo dnf install -y https://rpms.remirepo.net/enterprise/remi-release-10.rpm
sudo dnf config-manager --set-enabled remi-modular
sudo dnf module reset -y php
sudo dnf module list php
sudo dnf module install -y php:remi-8.4
```

A.8 «Peer authentication failed for user "suif"»

Faltó `-h 127.0.0.1` en el comando `psql` (Figura 5). Sin `-h`, psql usa el socket local y PostgreSQL compara tu usuario de Linux con el rol.

A.9 psql pide contraseña o «password authentication failed»

Revisa primero que `~/.pgpass` tenga permiso `600` y el formato `127.0.0.1:5432:*:suif:CLAVE`. Si la contraseña no coincide, genera una nueva y escríbela en los dos lugares:

VM · TERMINAL SSH

```
ls -l ~/.pgpass                    # debe decir -rw-------
cd /tmp
SUIF_BD_CLAVE="$(tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 24)"
sudo -u postgres psql -v ON_ERROR_STOP=1 -v clave="$SUIF_BD_CLAVE" <<'SQL'
ALTER ROLE suif PASSWORD :'clave';
SQL
printf '127.0.0.1:5432:*:suif:%s\n' "$SUIF_BD_CLAVE" > ~/.pgpass && chmod 600 ~/.pgpass
sed -i "s|^DB_PASSWORD=.*|DB_PASSWORD=$SUIF_BD_CLAVE|" "$SUIF_DIR/.env"
unset SUIF_BD_CLAVE
cd "$SUIF_DIR" && php artisan config:clear
```

Si dice *Ident authentication failed*, en `pg_hba.conf` quedó `ident`: repite el bloque de autenticación del paso 11.

A.10 «could not change directory to "/home/...": Permission denied»

Es sólo un aviso: el usuario `postgres` no puede entrar a tu carpeta personal. El comando sí se ejecuta. Para no verlo, haz `cd /tmp` antes de `sudo -u postgres …`.

A.11 «psql: error: …instalacion.sql: Permission denied»

Alguien corrió el script como `sudo -u postgres psql -f …`. El usuario `postgres` no puede leer archivos de tu proyecto y, además, si pudiera, las tablas quedarían a su nombre. Córrelo siempre como en el paso 17: con tu usuario, `-h 127.0.0.1 -U suif`.

A.12 La app dice «permission denied for table ...»

Las tablas quedaron a nombre de `postgres` por cargarlas con ese usuario. Pásalas a `suif`:

VM · TERMINAL SSH

```
cd /tmp
sudo -u postgres psql -d suif -Atc "SELECT format('ALTER TABLE public.%I OWNER TO suif;',
tablename)
  FROM pg_tables WHERE schemaname = 'public' AND tableowner <> 'suif'" | sudo -u postgres psql -d
suif
cd ~
```

A.13 «relation "metodo_pago" does not exist», «column ... does not exist» o 403 en Convocatorias

A la base le faltan partes del esquema: casi siempre se cargó desde una rama vieja con `suif_instalacion_completa.sql` o `suif_esquema_final.sql`, que estaban desfasados. Corre los scripts de `historico/` que faltan, del `10_comprobante_fiscal.sql` al `12_formato_pago.sql`: son idempotentes. O vuelve a la instantánea y usa `instalacion.sql`.

A.14 Se cargó suif_lleno.sql (de una rama vieja) y nadie puede iniciar sesión

Ese archivo trae los roles del esquema viejo. Lo más rápido es volver a la instantánea de VirtualBox. Si no tienes, recrea **sólo la base de tu VM**, nunca la del server: `cd /tmp && sudo -u postgres dropdb suif && sudo -u postgres createdb -O suif -E UTF8 -T template0 suif`, y repite los pasos 17 y 18.

A.15 Error 500 y «Permission denied» en storage/ o bootstrap/cache/

VM · TERMINAL SSH

```
cd "$SUIF_DIR"
sudo chown -R "$USER":apache storage bootstrap/cache
sudo find storage bootstrap/cache -type d -exec chmod 2775 {} +
sudo find storage bootstrap/cache -type f -exec chmod 0664 {} +
sudo restorecon -R "$SUIF_DIR"
php artisan optimize:clear
cat /etc/systemd/system/php-fpm.service.d/umask.conf   # debe decir UMask=0002
```

A.16 psql funciona en la terminal, pero el navegador dice «SQLSTATE[08006] ... Permission denied»

SELinux impide que Apache abra conexiones hacia la base: `sudo setsebool -P httpd_can_network_connect_db on`. Para ver qué bloqueó SELinux: `sudo ausearch -m avc -ts recent`.

A.17 Se ve la página de prueba de Apache, un 403 o el contenido de los archivos

El sitio no apunta a `public/`, sigue activo `welcome.conf` o faltan etiquetas de SELinux. Revisa el paso 15, `ls /etc/httpd/conf.d/` y `sudo restorecon -R "$SUIF_DIR"`.

A.18 La portada abre, pero /login u otra página da 404

Falta `AllowOverride All` en el `<Directory>` del paso 15, y Apache ignora `public/.htaccess`.

A.19 «Route [nombre] not defined» después de un pull

Quedó caché de rutas: `php artisan optimize:clear`.

A.20 «semanage: ... already defined»

Ya habías registrado esa regla de SELinux; no es un error. Sólo vuelve a ejecutar `sudo restorecon -R "$SUIF_DIR"`.

Anexo B — Diagnóstico rápido

Pega este bloque completo en la VM y comparte la salida con el equipo cuando pidas ayuda. No imprime contraseñas.

VM · TERMINAL SSH

```
source ~/suif-variables.sh
echo "== SISTEMA";    cat /etc/almalinux-release; getenforce; id
echo "== SERVICIOS";  systemctl is-active httpd php-fpm postgresql-18
echo "== VERSIONES";  php -v | head -1; composer --version 2>/dev/null
psql --version; node -v 2>/dev/null
echo "== PHP"
php -m | grep -Ei "^(pdo_pgsql|pdo_sqlite|gd|zip|intl|mbstring)$" | tr '\n' ' '; echo
echo "== GIT"
git -C "$SUIF_DIR" log --oneline -1
git -C "$SUIF_DIR" status -sb | head -1
ssh -T git@github.com 2>&1 | head -1
echo "== HTTP"
for u in / /login /persona/preregistro; do
  echo "$u $(curl -s -o /dev/null -w '%{http_code}' http://localhost$u)"
done
echo "== SELINUX"
ls -Zd "$SUIF_DIR" "$SUIF_DIR/storage"
getsebool httpd_can_network_connect_db
echo "== .env"
grep -E '^(APP_ENV|APP_URL|DB_HOST|DB_DATABASE|DB_USERNAME)=' "$SUIF_DIR/.env"
ls "$SUIF_DIR/bootstrap/cache/config.php" 2>/dev/null && echo "AVISO: configuración cacheada"
echo "== BASE DE DATOS"
psql -h 127.0.0.1 -U suif -d suif -Atc   "SELECT 'tablas: ' || count(*) FROM
information_schema.tables WHERE table_schema = 'public'"
psql -h 127.0.0.1 -U suif -d suif -Atc   "SELECT string_agg(rol_tipo_rol, ', ' ORDER BY
rol_id_rol) FROM rol"
echo "== ÚLTIMOS ERRORES"
sudo tail -n 5 /var/log/httpd/suif-error.log
grep -hE "ERROR|CRITICAL" "$SUIF_DIR/storage/logs/laravel-$(date +%F).log" 2>/dev/null | tail -5
```

Fuentes consultadas (octubre de 2026)

Tema	Fuente
AlmaLinux 10.2 e ISO mínima	almalinux.org/es/get-almalinux · repo.almalinux.org/almalinux/10/isos/x86_64
VirtualBox 7.2.20	virtualbox.org/wiki/Downloads · manual
PHP 8.4 en EL 10 (Remi)	rpms.remirepo.net/wizard (EL 10 · 8.4 · versión única)
PostgreSQL 18 en EL 10 (PGDG)	postgresql.org/download/linux/redhat
Composer	getcomposer.org/download
Node.js 24 LTS	nodejs.org/dist/latest-v24.x
Llaves SSH de GitHub	docs.github.com/es/authentication/connecting-to-github-with-ssh · huellas de GitHub
Scripts de la base de datos	`database/scripts/README.md` del repositorio

> **CONTACTO**
>
> Ante cualquier duda sobre los scripts SQL o el comportamiento de la aplicación, pregunta al equipo de SUIF antes de improvisar una solución. Y nunca pruebes nada de esta guía en el server de despliegue.